ZIPP

Product Requirements Document (PRD)

Green Brand System - Ride-Hailing + Delivery Platform

Market: Southwest Cameroon - Buea, Limbe, Kumba

Version 1.0 - MVP to Production Roadmap

Table of Contents

1. Executive Summary
2. Product Vision and Goals
3. Target Users and Personas
4. Scope and Feature Priorities
5. User Roles and Permissions
6. End-to-End User Flows
7. Rider Experience Requirements
8. Delivery Experience Requirements
9. Driver Experience Requirements
10. Admin Experience Requirements
11. Wallet and Payment Requirements
12. Pricing and Commission Model
13. Dispatch, Tracking and Real-Time Logic
14. Authentication, 2FA and Security
15. Backend, APIs and Data Model
16. Non-Functional Requirements
17. App Store and Deployment Readiness
18. Risks, Assumptions and Open Questions
19. MVP Acceptance Criteria
20. Roadmap

1. Executive Summary

Zipp is a cashless ride-hailing and on-demand delivery platform designed for African cities, starting with Buea and expanding to Limbe and Kumba. The app connects riders, senders and drivers through a secure wallet system, real-time tracking, driver matching, PIN-based completion and a controlled admin dashboard.

This PRD explains the product from the ground up: what Zipp is, who uses it, how money moves, how trips work, how delivery works, what the backend must provide, what screens must exist, and what needs to be true before the app is launched.

Brand principle: Simple, safe, local and cashless. The app must feel clean, reliable and easy for first-time users. The green color #00C853 is the primary action color across all screens.

Core product promise

Move and Deliver with Zipp: safe private rides, Standard or Express delivery, secure wallet payments, driver transparency and real-time tracking.

Success definition

- A new user can sign up, verify phone, enable 2FA and reach the home screen without confusion.
- A rider can top up wallet, book a ride, track a driver, complete with PIN and rate the driver.
- A sender can create a Standard or Express delivery, track it and complete with PIN.
- An approved driver can go online, accept requests, complete work and withdraw earnings.
- Admin can verify drivers, monitor trips, manage wallets, support users and detect fraud.

2. Product Vision and Goals

Vision

Zipp should become a trusted digital transport and delivery layer for local cities. It should reduce cash disputes, make fares clearer, give users live visibility, and create a reliable earning tool for drivers.

Business goals

- Launch first in a focused corridor in Buea, then expand gradually.
- Build a wallet-first model where Zipp earns 10-15% commission per completed ride or delivery.
- Create enough driver and rider liquidity to reach 1,000 rides per day over time.
- Provide a scalable structure that can later support separate rider and driver apps without losing user data.

Product goals

- One app at MVP: Rider, Driver and Admin access are role-based.
- Delivery is one feature only, with Standard and Express options. No separate Logistics feature.
- Top Up and Withdraw screens must not sit openly on the home/profile screen. They appear only when triggered.
- Every item that looks clickable must be clickable and must navigate, open a modal or perform an action.

Non-goals for MVP

- No cash payment option in MVP.
- No separate Logistics tab.
- No vehicle marketplace, food delivery or e-commerce marketplace in MVP.
- No complex multi-stop delivery in MVP.

3. Target Users and Personas

Persona	Needs	Pain Points	Zipp Solution
Rider	Book private rides, see price, avoid bargaining.	Cash disputes, unreliable pickups, unsafe trips.	Wallet payment, driver tracking, PIN completion, ratings.
Sender	Send packages to another person quickly.	Unclear delivery timing, no proof of delivery.	Standard/Express delivery, recipient details, tracking link, PIN confirmation.
Driver	Earn reliably and withdraw earnings.	Unclear payments, wasted time, fake trips.	Online status, trip requests, wallet earnings, ratings, withdrawal flow.
Admin/Support	Operate the platform and resolve issues.	Fraud, payment mismatch, bad drivers, user complaints.	Admin dashboard, verification, wallet adjustment, support tickets, fraud flags.

Local context

- Users may have unstable internet; the app must show clear loading, retry and failed states.
- Mobile money is central, but API confirmations can be delayed; top-ups must support pending states and webhooks.
- Drivers may go offline or lose GPS; the system must use heartbeat, last known location and trip recovery logic.

4. Scope and Feature Priorities

Priority	Feature	Description
P0	Authentication + 2FA	Signup, login, phone OTP, 2FA, password reset and secure session management.
P0	Wallet	Top-up, frozen funds, withdrawals, transaction history and refunds.
P0	Ride booking	Fare estimate, driver matching, trip tracking, PIN completion and rating.
P0	Delivery	Standard or Express delivery only; recipient, package notes, tracking and PIN completion.
P0	Driver app	Online/offline, accept/decline, complete trips, wallet, earnings and ratings.
P0	Admin dashboard	Users, drivers, verification, trips, wallets, support and fraud alerts.
P1	Push notifications	Trip updates, payment updates, driver requests and support messages.
P1	Referral system	Invite users and earn wallet credits.
P2	Separate rider/driver apps	Split later while keeping same backend and accounts.

MVP principle

The MVP should be complete enough to operate real trips, but not overbuilt. Standard/Express delivery, ride booking, wallet, driver acceptance and admin operations are mandatory. Complex freight, multi-stop logistics and non-cash payments are postponed.

5. User Roles and Permissions

Zipp uses one account system. A user may have more than one role, but access must be controlled by the backend, not just by hiding buttons in the UI.

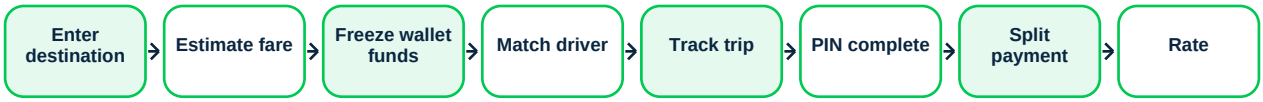
Role	Access	Restrictions
Rider	Book rides, send deliveries, wallet, support, ratings.	Cannot access driver mode unless approved.
Driver	Driver dashboard, accept requests, earnings, withdrawal, vehicle/documents.	Cannot go online until verified by admin.
Admin	Dashboard, users, drivers, wallets, support, reports, fraud tools.	Must pass admin login + 2FA; not visible to normal users.
Support staff	Support tickets and limited user/trip view.	No wallet adjustment unless granted.
Finance admin	Wallet transactions, withdrawals, reconciliations.	No driver verification unless granted.

Role-switch rules

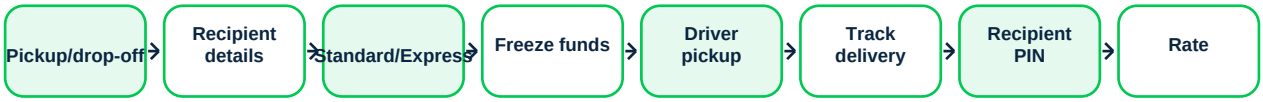
- Switch to Driver appears only after user has a driver profile.
- If driver profile is pending, show status: Under Review.
- If rejected, show rejection reason and resubmit option.
- Open Admin Dashboard appears only for ADMIN users; never for normal riders.

6. End-to-End User Flows

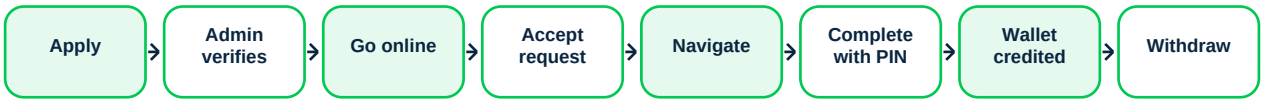
Ride flow



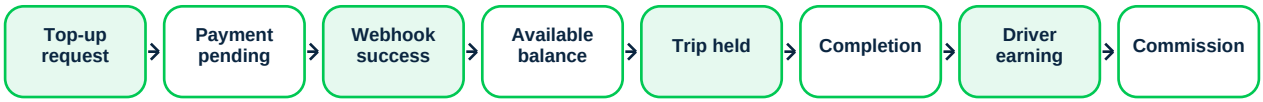
Delivery flow



Driver flow



Payment flow



7. Rider Experience Requirements

Rider home

- Header with greeting and notification icon.
- Wallet summary card showing available and frozen balance.
- Search bar: Where to?
- Primary actions: Ride, Delivery, Wallet, Support.
- Saved places: Home, Work, frequently used locations.
- Bottom navigation: Home, Activity, Wallet, Profile.

Ride booking screens

- Destination entry screen: pickup and destination fields.
- Fare estimate screen with Classic, Comfort and Comfort Plus.
- Confirmation screen must show fare, wallet balance and insufficient balance warning if needed.
- Finding driver screen must have cancel option and loading state.
- Driver assigned screen shows driver name, rating, vehicle, plate, ETA, call, chat and cancel.
- Trip in progress screen shows route, ETA, fare, share trip and emergency option.
- Trip complete screen shows fare paid from wallet and next step: Rate Driver.

Rider profile

Profile items such as Personal Info, Documents, Settings, Support and Security & 2FA must be clickable list rows. They must open separate screens. They must never be dead UI.

8. Delivery Experience Requirements

Delivery is one product feature. It must not be duplicated as Logistics. The user chooses delivery speed: Standard or Express.

Delivery Mode	Meaning	Pricing
Standard Delivery	Affordable normal delivery for non-urgent items.	1.0x delivery fare
Express Delivery	Priority delivery for faster arrival.	1.5x delivery fare

Delivery screens

- Delivery start: user enters pickup and drop-off.
- Recipient details: recipient name and phone.
- Package notes: optional notes, item description and handling instructions.
- Mode selection: Standard or Express with price/time description.
- Price estimate: pickup, drop-off, mode, estimated distance, price and wallet status.
- Driver assigned: driver details, ETA and chat.
- Tracking: live route and share tracking link.
- Completion: recipient gives PIN; delivery marked completed; payment split.
- Rate delivery: sender rates the delivery experience.

Delivery edge cases

- Recipient unreachable: show call recipient, retry delivery or return-to-sender workflow.
- Sender cancels before driver accepts: release held funds.
- Sender cancels after pickup: require support/admin resolution.
- PIN refused or forgotten: create dispute and allow admin override after evidence review.

9. Driver Experience Requirements

Driver onboarding

- Driver applies from app with personal info, vehicle details and document upload.
- Driver cannot go online until admin approves.
- Driver status can be Pending, Approved, Rejected, Suspended.

Driver home

- Online/offline toggle at the top.
- Wallet balance and today’s earnings must be visible.
- Stats: total trips, rating, acceptance rate.
- New trip request screen includes route preview, pickup, drop-off, fare, distance, Accept and Decline buttons.
- Accept button locks driver assignment with database transaction so multiple drivers cannot win same trip.
- Trip completion requires PIN entry.

Driver earnings and wallet

- Earnings dashboard with Day, Week and Month tabs.
- Driver wallet balance and transaction history.
- Withdraw button opens withdrawal screen only when tapped.
- Withdrawal request can be pending, approved, rejected or paid.

10. Admin Experience Requirements

Admin is the control center. Admin access must be protected by role and 2FA. Admin screens must be separate from normal app screens except for an admin-only entry point visible only to admin accounts.

Admin Area	Requirements
Dashboard	Total users, drivers, trips, deliveries, revenue, pending verifications, active issues.
Users	Search, filter, view profile, ban, shadow-ban, view wallet and activity.
Drivers	Verify/reject, review documents, suspend, view ratings, vehicle data.
Trips	Monitor active/completed/cancelled trips, logs, payments and disputes.
Deliveries	Monitor delivery status, recipient details and completion evidence.
Wallet	View ledger, top-up status, frozen funds, refunds, manual adjustments.
Withdrawals	Approve/reject driver withdrawals; record payment status.
Support	View tickets, assign staff, reply and resolve.
Fraud alerts	Flag offline trips, repeated cancellations, PIN issues and suspicious wallet activity.
Settings	Commission rate, pricing rules, payment method enable/disable.

Admin seed account

For development only: admin@zipp.app / ZippAdmin123!. Production must force password change and never hardcode this credential in the app.

11. Wallet and Payment Requirements

The wallet is the most important financial component. It must be ledger-based. Do not directly edit balances as a single number. Every money movement must create a ledger record.

Wallet states

- Available Balance: funds the user can spend or withdraw.
- Frozen Balance: funds currently held for an active ride or delivery.

Transaction types

Type	Meaning
TOP_UP	User adds money through MTN MoMo or Orange Money.
PAYMENT	User pays for a ride or delivery.
COMMISSION	Zipp commission taken from completed payment.
DRIVER_EARNING	Driver portion of completed fare.
WITHDRAWAL	Driver/user withdrawal to mobile money.
REFUND	Held or paid funds returned to user.
ADMIN_ADJUSTMENT	Manual correction by authorized admin.

Money flow

- Top-up begins as PENDING.
- Webhook confirms payment as SUCCESS, then wallet becomes available.
- When user requests ride/delivery, fare amount is moved to HELD/frozen.
- On PIN completion, held amount becomes successful payment; driver receives 85-90%, Zipp receives 10-15%.
- If request fails or is cancelled under valid rules, held amount is refunded.

Payment providers

- MTN MoMo provider integration.
- Orange Money provider integration.
- Mock payment provider for development and testing.
- Real API credentials must be stored in environment variables, never in code.

12. Pricing and Commission Model

Pricing Area	Rule
Ride Classic	1.0x multiplier
Ride Comfort	1.3x multiplier
Ride Comfort Plus	1.7x multiplier
Delivery Standard	1.0x multiplier
Delivery Express	1.5x multiplier
Night 21:00-00:00	1.2x multiplier
Night 00:00-06:00	1.5x multiplier
Zipp commission	10-15% configurable by admin

Fare calculation

Fare should use a base fare + distance rate + applicable multipliers. Admin must be able to update pricing rules without app redeployment.

Revenue example

Assumption	Value
Average fare	1,300 XAF
Daily trips	1,000
10% commission	130,000 XAF per day
15% commission	195,000 XAF per day
Monthly range	3,900,000 - 5,850,000 XAF

13. Dispatch, Tracking and Real-Time Logic

Dispatch model

- Find approved online drivers within 3km.
- Rank drivers by distance, rating, acceptance rate and activity.
- Send request to top drivers, not every driver in the city.
- First accepted driver wins using database transaction to avoid duplicate assignment.
- If no one accepts, expand radius to 5km and retry.

Driver location

- Driver app sends location updates every 3-5 seconds while online or active.
- Backend stores latest location in DriverLocation.
- Realtime updates are sent through Socket.IO.
- If network drops, show last known location and sync when back online.

Trip logs

Every major trip/delivery action must be logged: request, accept, arrive, start, complete, cancel, refund, admin override. Logs must include timestamp, actor and location when available.

14. Authentication, 2FA and Security

Authentication requirements

- Register with phone number and password.
- Verify phone with OTP.
- Login with password and then 2FA if enabled/required.
- Forgot password and reset flow with OTP.
- Refresh token support for long sessions.
- Secure token storage on mobile.

2FA requirements

- 2FA applies to all users.
- Users see Enable 2FA, Verify 2FA and Backup Codes screens.
- Admin must always use 2FA.
- Backup codes must be one-time use and stored hashed.

Security requirements

- bcrypt password hashing.
- JWT access tokens with refresh tokens.
- Rate limit login, OTP and password reset endpoints.
- Validate all inputs.
- Role-based access control on backend.
- Admin action logs.
- No raw passwords, no secrets in mobile code, no hardcoded production credentials.

15. Backend, APIs and Data Model

Recommended stack

- Mobile: React Native Expo + TypeScript.
- Backend: Node.js + Express or NestJS + TypeScript.
- Database: PostgreSQL + PostGIS.
- ORM: Prisma.
- Queues: Redis + BullMQ.
- Realtime: Socket.IO.
- Admin: React web dashboard or responsive admin module.

Core API groups

Group	Sample Endpoints
Auth	POST /auth/register, /auth/login, /auth/verify-phone, /auth/forgot-password, /auth/reset-password, /auth/enable-2fa
Wallet	GET /wallet, GET /wallet/transactions, POST /wallet/top-up, POST /wallet/withdraw, POST /wallet/refund
Trips	POST /trips/request, /trips/estimate, /trips/accept, /trips/start, /trips/complete, /trips/cancel
Deliveries	POST /deliveries/request, /estimate, /accept, /pickup, /complete, /cancel
Driver	POST /driver/apply, /go-online, /go-offline, GET /driver/earnings, /ratings
Admin	GET /admin/dashboard, /users, /drivers, /trips, /wallet-transactions, /support-tickets, /fraud-alerts

Database models

User, UserRole, DriverProfile, Vehicle, DriverDocument, Wallet, WalletLedger, Trip, TripLog, Delivery, DeliveryLog, DriverLocation, Rating, SupportTicket, SupportMessage, AdminActionLog, PricingRule, WithdrawalRequest, OTPCode, TwoFactorAuth, RefreshToken.

16. Non-Functional Requirements

Category	Requirement
Performance	Common API responses under 500ms; dispatch begins within 5 seconds.
Reliability	Handle payment delays, network drops, failed webhooks and trip recovery.
Scalability	Architecture should support growth from closed beta to 10,000+ daily trips.
Availability	Target 99% uptime for MVP production.
Security	Protected APIs, 2FA, token security, rate limiting and audit logs.
Usability	Simple flows, clear states, no dead buttons, no confusing duplicate features.
Observability	Logs, error tracking, admin alerts and payment reconciliation reports.

Offline and weak network behavior

- Show retry buttons instead of infinite loaders.
- Queue location updates locally if connection drops during active trip.
- Payment status should show Pending until confirmed.
- Do not show wallet balance as credited until webhook verification succeeds.

17. App Store and Deployment Readiness

Mobile readiness

- App icon and splash screen configured.
- Bundle identifiers and package names set.
- Environment configuration for development, staging and production.
- Privacy Policy and Terms links.
- Expo EAS build setup or equivalent native build setup.
- Push notification setup.
- No test credentials or secrets in production app.

Backend readiness

- Production database configured.
- Prisma migrations applied.
- Redis configured.
- Payment webhook URLs public and secured.
- Admin seed account replaced or forced password change.
- Monitoring and backups in place.

Release checklist

- Signup/login tested.
- Top-up/payment pending/success/failed tested.
- Ride request/accept/complete/refund tested.
- Delivery Standard/Express tested.
- Admin verification and wallet adjustment tested.
- All clickable UI elements tested.

18. Risks, Assumptions and Open Questions

Risk	Impact	Mitigation
MoMo/Orange API delays	Wallet balances may be wrong if credited too early.	Use pending states, webhooks and reconciliation.
Low driver supply	Riders may not find drivers.	Start in focused corridor; incentives; dispatch radius expansion.
Network instability	Tracking and trip updates may fail.	Realtime fallback, last known location, retry logic.
Fraud/offline trips	Revenue leakage and user disputes.	PIN completion, trip logs, fraud flags, admin review.
Poor UI interactions	Users get stuck.	No dead buttons; every list item navigates.
Admin abuse	Wallet or user data misuse.	Role permissions and admin action logs.

Open questions

- Exact local pricing per route and per kilometer.
- Commission: should default be 10%, 12% or 15% at launch?
- Who handles support during beta and what hours?
- Will driver withdrawals be automatic or admin-approved at first?
- Which map provider will be used for production?

19. MVP Acceptance Criteria

The MVP is acceptable only when these conditions are met.

Area	Acceptance Criteria
Auth	User can register, verify phone, login, reset password and pass 2FA.
Wallet	User can top up, see pending/success, hold funds, refund and see transactions.
Ride	Rider can book, driver can accept, trip can start and complete with PIN.
Delivery	Sender can choose Standard/Express, track and complete with PIN.
Driver	Approved driver can go online, accept, complete, view earnings and withdraw.
Admin	Admin can login, verify driver, view trips, support tickets and wallet transactions.
UI	No dead buttons; Top Up/Withdraw are hidden until tapped; Delivery is not duplicated.
Security	Passwords hashed, tokens secure, roles enforced, admin 2FA enabled.

20. Roadmap

Phase	Goal	Deliverables
Phase 1 - Closed Beta	Validate real ride and delivery flows in Buea corridor.	Core app, wallet mock/live sandbox, driver verification, admin dashboard.
Phase 2 - Payment Hardening	Stabilize real money flows.	MTN/Orange production credentials, webhooks, reconciliation, withdrawal process.
Phase 3 - Growth	Increase usage and driver supply.	Referral credits, driver incentives, improved support tools.
Phase 4 - Scale	Expand to Limbe and Kumba.	Better dispatch scoring, heatmaps, admin analytics, multi-city settings.
Phase 5 - App Split	Separate Rider and Driver apps if needed.	Same backend and database; separate app interfaces.

Final note

Zipp should not be treated as just an app UI. It is a transport and delivery operating system. The strongest version of Zipp is simple for users, strict with money, transparent with drivers, and controlled by a powerful admin layer.